E-COMMERCE 360°

SQL QUERIES DOCUMENTATION

MySQL | Database Validation | Sales Analytics | Customer RFM

Faithful reproduction of the uploaded ecommerce_360.sql file. Original query order and SQL text are preserved; headings and line ranges are editorial navigation aids. Queries have not been executed or corrected.

Contents

1. Database Setup and Table Checks (source lines 1–46)
2. Record Counts and Data Quality Checks (source lines 47–179)
3. Referential Integrity Checks (source lines 180–219)
4. Sales and KPI Analysis (source lines 220–289)
5. Monthly, Category, Product and Regional Analysis (source lines 290–361)
6. Customer and Payment Analysis (source lines 362–431)
7. Year-over-Year Growth and RFM Analysis (source lines 432–633)
8. Sales Analysis View and Final Verification (source lines 634–684)

01 Database Setup and Table Checks

Original source lines 1–46

```
CREATE DATABASE ecommerce_360;

USE ecommerce_360;

SELECT DATABASE();

USE ecommerce_360;

SELECT * FROM customers;

SELECT COUNT(*) AS Total_Customers
FROM customers;

USE ecommerce_360;

SELECT * FROM customers;

USE ecommerce_360;

SELECT * FROM products;

USE ecommerce_360;
USE ecommerce_360;

SELECT * FROM order_details;

SELECT COUNT(*) AS Total_Order_Details
FROM order_details;

SELECT * FROM orders;

SELECT COUNT(*) AS Total_Orders
FROM orders;

SELECT COUNT(*) AS Total_Products
FROM products;

USE ecommerce_360;

SELECT * FROM payments;

SELECT COUNT(*) AS Total_Payments
FROM payments;
```

02 Record Counts and Data Quality Checks

Original source lines 47–179

```
USE ecommerce_360;

SELECT 'Customers' AS Table_Name,
COUNT(*) AS Total_Records
FROM customers

UNION ALL

SELECT 'Products', COUNT(*)
FROM products

UNION ALL

SELECT 'Orders', COUNT(*)
FROM orders

UNION ALL

SELECT 'Order Details', COUNT(*)
FROM order_details

UNION ALL
SELECT 'Payments', COUNT(*)
FROM payments;

USE ecommerce_360;

SELECT
    COUNT(*) AS Total_Customers,

    SUM(customer_id IS NULL) AS Missing_ID,

    SUM(customer_name IS NULL) AS Missing_Name,

    SUM(city IS NULL) AS Missing_City,

    SUM(state IS NULL) AS Missing_State,

    SUM(signup_date IS NULL) AS Missing_Date

FROM customers;

USE ecommerce_360;
SELECT
    COUNT(*) AS Total_Products,
    SUM(product_id IS NULL) AS Missing_ID,
    SUM(product_name IS NULL) AS Missing_Name,
    SUM(category IS NULL) AS Missing_Category,
    SUM(unit_price IS NULL) AS Missing_Price,
    SUM(unit_cost IS NULL) AS Missing_Cost
FROM products;

USE ecommerce_360;

SELECT
    COUNT(*) AS Total_Orders,
    SUM(order_id IS NULL) AS Missing_Order_ID,
    SUM(customer_id IS NULL) AS Missing_Customer_ID,
    SUM(order_date IS NULL) AS Missing_Order_Date,
    SUM(order_status IS NULL) AS Missing_Order_Status
FROM orders;

USE ecommerce_360;
SELECT
    COUNT(*) AS Total_Order_Details,
    SUM(order_detail_id IS NULL) AS Missing_Detail_ID,
```

```
    SUM(order_id IS NULL) AS Missing_Order_ID,
    SUM(product_id IS NULL) AS Missing_Product_ID,
    SUM(quantity IS NULL) AS Missing_Quantity,
    SUM(unit_price IS NULL) AS Missing_Price,
    SUM(discount_pct IS NULL) AS Missing_Discount,
    SUM(sales_amount IS NULL) AS Missing_Sales,
    SUM(profit_amount IS NULL) AS Missing_Profit
FROM order_details;

USE ecommerce_360;

SELECT
    COUNT(*) AS Total_Payments,
    SUM(payment_id IS NULL) AS Missing_Payment_ID,
    SUM(order_id IS NULL) AS Missing_Order_ID,
    SUM(payment_method IS NULL) AS Missing_Method,
    SUM(payment_status IS NULL) AS Missing_Status,
    SUM(payment_amount IS NULL) AS Missing_Amount
FROM payments;
USE ecommerce_360;

SELECT
    customer_id,
    COUNT(*) AS Duplicate_Count
FROM customers
GROUP BY customer_id
HAVING COUNT(*) > 1;

SELECT
    product_id,
    COUNT(*) AS Duplicate_Count
FROM products
GROUP BY product_id
HAVING COUNT(*) > 1;

USE ecommerce_360;

SELECT
    order_id,
    COUNT(*) AS Duplicate_Count
FROM orders
GROUP BY order_id
HAVING COUNT(*) > 1;

USE ecommerce_360;

SELECT
    order_detail_id,
    COUNT(*) AS Duplicate_Count
FROM order_details
GROUP BY order_detail_id
HAVING COUNT(*) > 1;

USE ecommerce_360;

SELECT
    payment_id,
    COUNT(*) AS Duplicate_Count
FROM payments
GROUP BY payment_id
HAVING COUNT(*) > 1;
```

03 Referential Integrity Checks

Original source lines 180–219

```
USE ecommerce_360;

SELECT
    COUNT(*) AS Invalid_Customer_References
FROM orders o
LEFT JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.customer_id IS NULL;


USE ecommerce_360;

SELECT
    COUNT(*) AS Invalid_Product_References
FROM order_details od
LEFT JOIN products p
    ON od.product_id = p.product_id
WHERE p.product_id IS NULL;


USE ecommerce_360;
SELECT
    COUNT(*) AS Invalid_Order_References
FROM order_details od
LEFT JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_id IS NULL;


USE ecommerce_360;

SELECT
    COUNT(*) AS Invalid_Payment_References
FROM payments p
LEFT JOIN orders o
    ON p.order_id = o.order_id
WHERE o.order_id IS NULL;
```

04 Sales and KPI Analysis

Original source lines 220–289

```
USE ecommerce_360;

SELECT
    ROUND(SUM(od.sales_amount), 2) AS Total_Revenue
FROM order_details od
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered';

USE ecommerce_360;

SELECT
    ROUND(SUM(od.profit_amount), 2) AS Total_Profit
FROM order_details od
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered';

USE ecommerce_360;

SELECT
    ROUND(SUM(od.sales_amount), 2) AS Total_Revenue,
    ROUND(SUM(od.profit_amount), 2) AS Total_Profit,
    ROUND(SUM(od.sales_amount - od.profit_amount), 2) AS Total_Cost
FROM order_details od
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered';


USE ecommerce_360;

SELECT
    ROUND(
        SUM(od.profit_amount) * 100.0 /
        NULLIF(SUM(od.sales_amount), 0),
        2
    ) AS Profit_Margin_Percentage
FROM order_details od
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered';

USE ecommerce_360;
SELECT
    COUNT(DISTINCT order_id) AS Total_Delivered_Orders
FROM orders
WHERE order_status = 'Delivered';

USE ecommerce_360;

SELECT
    ROUND(
        SUM(od.sales_amount) /
        NULLIF(COUNT(DISTINCT o.order_id), 0),
        2
    ) AS Average_Order_Value
FROM orders o
JOIN order_details od
    ON o.order_id = od.order_id
WHERE o.order_status = 'Delivered';

USE ecommerce_360;

SELECT
    COUNT(DISTINCT customer_id) AS Total_Active_Customers
```

```
FROM orders
WHERE order_status = 'Delivered';
```

05 Monthly, Category, Product and Regional Analysis

Original source lines 290–361

```sql
USE ecommerce_360;

SELECT
    YEAR(o.order_date) AS Sales_Year,
    MONTH(o.order_date) AS Month_Number,
    MONTHNAME(o.order_date) AS Sales_Month,
    ROUND(SUM(od.sales_amount), 2) AS Total_Sales
FROM orders o
JOIN order_details od
    ON o.order_id = od.order_id
WHERE o.order_status = 'Delivered'
GROUP BY
    YEAR(o.order_date),
    MONTH(o.order_date),
    MONTHNAME(o.order_date)
ORDER BY Sales_Year, Month_Number;

USE ecommerce_360;

SELECT
    p.category AS Category,
    ROUND(SUM(od.sales_amount), 2) AS Total_Sales,
    ROUND(SUM(od.profit_amount), 2) AS Total_Profit,
    SUM(od.quantity) AS Total_Quantity
FROM order_details od
JOIN products p
    ON od.product_id = p.product_id
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered'
GROUP BY p.category
ORDER BY Total_Sales DESC;


USE ecommerce_360;

SELECT
    p.product_name AS Product_Name,
    p.category AS Category,
    SUM(od.quantity) AS Total_Quantity,
    ROUND(SUM(od.sales_amount), 2) AS Total_Sales,
    ROUND(SUM(od.profit_amount), 2) AS Total_Profit
FROM order_details od
JOIN products p
    ON od.product_id = p.product_id
JOIN orders o
    ON od.order_id = o.order_id
WHERE o.order_status = 'Delivered'
GROUP BY
    p.product_id,
    p.product_name,
    p.category
ORDER BY Total_Sales DESC
LIMIT 10;

USE ecommerce_360;

SELECT
    c.state AS State,
    COUNT(DISTINCT o.order_id) AS Total_Orders,
    COUNT(DISTINCT c.customer_id) AS Total_Customers,
    ROUND(SUM(od.sales_amount), 2) AS Total_Sales,
    ROUND(SUM(od.profit_amount), 2) AS Total_Profit
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
```

```
JOIN order_details od
    ON o.order_id = od.order_id
WHERE o.order_status = 'Delivered'
GROUP BY c.state
ORDER BY Total_Sales DESC;
```

06 Customer and Payment Analysis

Original source lines 362–431

```sql
USE ecommerce_360;

SELECT
    c.customer_id,
    c.customer_name,
    c.city,
    c.state,
    COUNT(DISTINCT o.order_id) AS Total_Orders,
    ROUND(SUM(od.sales_amount), 2) AS Total_Spending
FROM customers c
JOIN orders o
    ON c.customer_id = o.customer_id
JOIN order_details od
    ON o.order_id = od.order_id
WHERE o.order_status = 'Delivered'
GROUP BY
    c.customer_id,
    c.customer_name,
    c.city,
    c.state
ORDER BY Total_Spending DESC
LIMIT 10;
```

```sql
USE ecommerce_360;

SELECT
    Customer_Type,
    COUNT(*) AS Total_Customers
FROM (
    SELECT
        customer_id,
        CASE
            WHEN COUNT(DISTINCT order_id) = 1
                THEN 'One-Time Customer'
            ELSE 'Returning Customer'
        END AS Customer_Type
    FROM orders
    WHERE order_status = 'Delivered'
    GROUP BY customer_id
) AS Customer_Analysis
GROUP BY Customer_Type;
```

```sql
USE ecommerce_360;
SELECT
    p.payment_method AS Payment_Method,
    COUNT(DISTINCT p.order_id) AS Total_Orders,
    ROUND(SUM(p.payment_amount), 2) AS Total_Amount
FROM payments p
JOIN orders o
    ON p.order_id = o.order_id
WHERE o.order_status = 'Delivered'
AND p.payment_status = 'Paid'
GROUP BY p.payment_method
ORDER BY Total_Orders DESC;
```

```sql
USE ecommerce_360;

SELECT
    order_status AS Order_Status,
    COUNT(*) AS Total_Orders,
    ROUND(
        COUNT(*) * 100.0 /
        (SELECT COUNT(*) FROM orders),
        2
    ) AS Order_Percentage
FROM orders
GROUP BY order_status
```

```
ORDER BY Total_Orders DESC;
```

07 Year-over-Year Growth and RFM Analysis

Original source lines 432–633

```
USE ecommerce_360;

WITH Yearly_Sales AS (
    SELECT
        YEAR(o.order_date) AS Sales_Year,
        ROUND(SUM(od.sales_amount), 2) AS Total_Sales
    FROM orders o
    JOIN order_details od
        ON o.order_id = od.order_id
    WHERE o.order_status = 'Delivered'
    GROUP BY YEAR(o.order_date)
)

SELECT
    Sales_Year,
    Total_Sales,
    LAG(Total_Sales) OVER (
        ORDER BY Sales_Year
    ) AS Previous_Year_Sales,

    ROUND(
        (Total_Sales -
        LAG(Total_Sales) OVER (ORDER BY Sales_Year))
        * 100.0 /
        NULLIF(
            LAG(Total_Sales) OVER (ORDER BY Sales_Year), 0
        ),
        2
    ) AS Growth_Percentage

FROM Yearly_Sales;

USE ecommerce_360;

SELECT
    c.customer_id,
    c.customer_name,

    DATEDIFF(
        '2026-01-01',
        MAX(o.order_date)
    ) AS Recency_Days,

    COUNT(DISTINCT o.order_id) AS Frequency,
    ROUND(
        SUM(od.sales_amount), 2
    ) AS Monetary_Value

FROM customers c

JOIN orders o
    ON c.customer_id = o.customer_id

JOIN order_details od
    ON o.order_id = od.order_id

WHERE o.order_status = 'Delivered'

GROUP BY
    c.customer_id,
    c.customer_name

ORDER BY Monetary_Value DESC;
WITH RFM_Base AS (
    SELECT
        c.customer_id,
```

```
        c.customer_name,
        DATEDIFF('2026-01-01', MAX(o.order_date)) AS Recency,
        COUNT(DISTINCT o.order_id) AS Frequency,
        ROUND(SUM(od.sales_amount), 2) AS Monetary
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_details od ON o.order_id = od.order_id
    WHERE o.order_status = 'Delivered'
    GROUP BY c.customer_id, c.customer_name
)

SELECT
    *,
    NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
    NTILE(5) OVER (ORDER BY Frequency ASC) AS F_Score,
    NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
FROM RFM_Base;
USE ecommerce_360;

WITH RFM_Base AS (
    SELECT
        c.customer_id,
        c.customer_name,
        DATEDIFF('2026-01-01', MAX(o.order_date)) AS Recency,
        COUNT(DISTINCT o.order_id) AS Frequency,
        ROUND(SUM(od.sales_amount), 2) AS Monetary
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_details od ON o.order_id = od.order_id
    WHERE o.order_status = 'Delivered'
    GROUP BY c.customer_id, c.customer_name
),

RFM_Scores AS (
    SELECT *,
        NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
        NTILE(5) OVER (ORDER BY Frequency ASC, customer_id) AS F_Score,
        NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
    FROM RFM_Base
)

SELECT *,
    CASE
        WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4
            THEN 'VIP Customers'

        WHEN F_Score >= 4
            THEN 'Loyal Customers'

        WHEN R_Score <= 2 AND F_Score >= 3
            THEN 'At Risk Customers'

        WHEN R_Score >= 4 AND F_Score <= 2
            THEN 'Promising Customers'

        ELSE 'Regular Customers'
    END AS Customer_Segment

FROM RFM_Scores;
SELECT
    Customer_Segment,
    COUNT(*) AS Total_Customers
FROM (
    SELECT *,
        CASE
            WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4
                THEN 'VIP Customers'
            WHEN F_Score >= 4
                THEN 'Loyal Customers'
            WHEN R_Score <= 2 AND F_Score >= 3
                THEN 'At Risk Customers'
```

```
            WHEN R_Score >= 4 AND F_Score <= 2
                THEN 'Promising Customers'
            ELSE 'Regular Customers'
        END AS Customer_Segment
    FROM RFM_Scores
) AS Segmented_Customers
GROUP BY Customer_Segment
ORDER BY Total_Customers DESC;
```

```
USE ecommerce_360;

WITH RFM_Base AS (
    SELECT
        c.customer_id,
        c.customer_name,
        DATEDIFF('2026-01-01', MAX(o.order_date)) AS Recency,
        COUNT(DISTINCT o.order_id) AS Frequency,
        ROUND(SUM(od.sales_amount), 2) AS Monetary
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    JOIN order_details od ON o.order_id = od.order_id
    WHERE o.order_status = 'Delivered'
    GROUP BY c.customer_id, c.customer_name
),

RFM_Scores AS (
    SELECT *,
        NTILE(5) OVER (ORDER BY Recency DESC) AS R_Score,
        NTILE(5) OVER (ORDER BY Frequency ASC, customer_id) AS F_Score,
        NTILE(5) OVER (ORDER BY Monetary ASC) AS M_Score
    FROM RFM_Base
),

Customer_Segments AS (
    SELECT *,
        CASE
            WHEN R_Score >= 4 AND F_Score >= 4 AND M_Score >= 4
                THEN 'VIP Customers'
            WHEN F_Score >= 4
                THEN 'Loyal Customers'
            WHEN R_Score <= 2 AND F_Score >= 3
                THEN 'At Risk Customers'
            WHEN R_Score >= 4 AND F_Score <= 2
                THEN 'Promising Customers'
            ELSE 'Regular Customers'
        END AS Customer_Segment
    FROM RFM_Scores
)

SELECT
    Customer_Segment,
    COUNT(*) AS Total_Customers
FROM Customer_Segments
GROUP BY Customer_Segment
ORDER BY Total_Customers DESC;
```

08 Sales Analysis View and Final Verification

Original source lines 634–684

```
USE ecommerce_360;

CREATE OR REPLACE VIEW vw_sales_analysis AS

SELECT
    o.order_id,
    o.order_date,
    o.order_status,

    c.customer_id,
    c.customer_name,
    c.city,
    c.state,

    p.product_id,
    p.product_name,
    p.category,

    od.quantity,
    od.unit_price,
    od.discount_pct,
    od.sales_amount,
    od.profit_amount

FROM order_details od

JOIN orders o
    ON od.order_id = o.order_id

JOIN customers c
    ON o.customer_id = c.customer_id

JOIN products p
    ON od.product_id = p.product_id;

    SELECT *
FROM vw_sales_analysis
LIMIT 10;

SELECT
    COUNT(*) AS total_rows,
    SUM(sales_amount) AS total_sales
FROM ecommerce_360.order_details;
SELECT
    COUNT(DISTINCT o.order_id) AS delivered_orders,
    ROUND(SUM(od.sales_amount), 2) AS delivered_revenue
FROM ecommerce_360.orders o
JOIN ecommerce_360.order_details od
    ON o.order_id = od.order_id
WHERE o.order_status = 'Delivered';
```